DAY 1

Large Language Model

LLM 동작원리 · Agent · RAG · Application

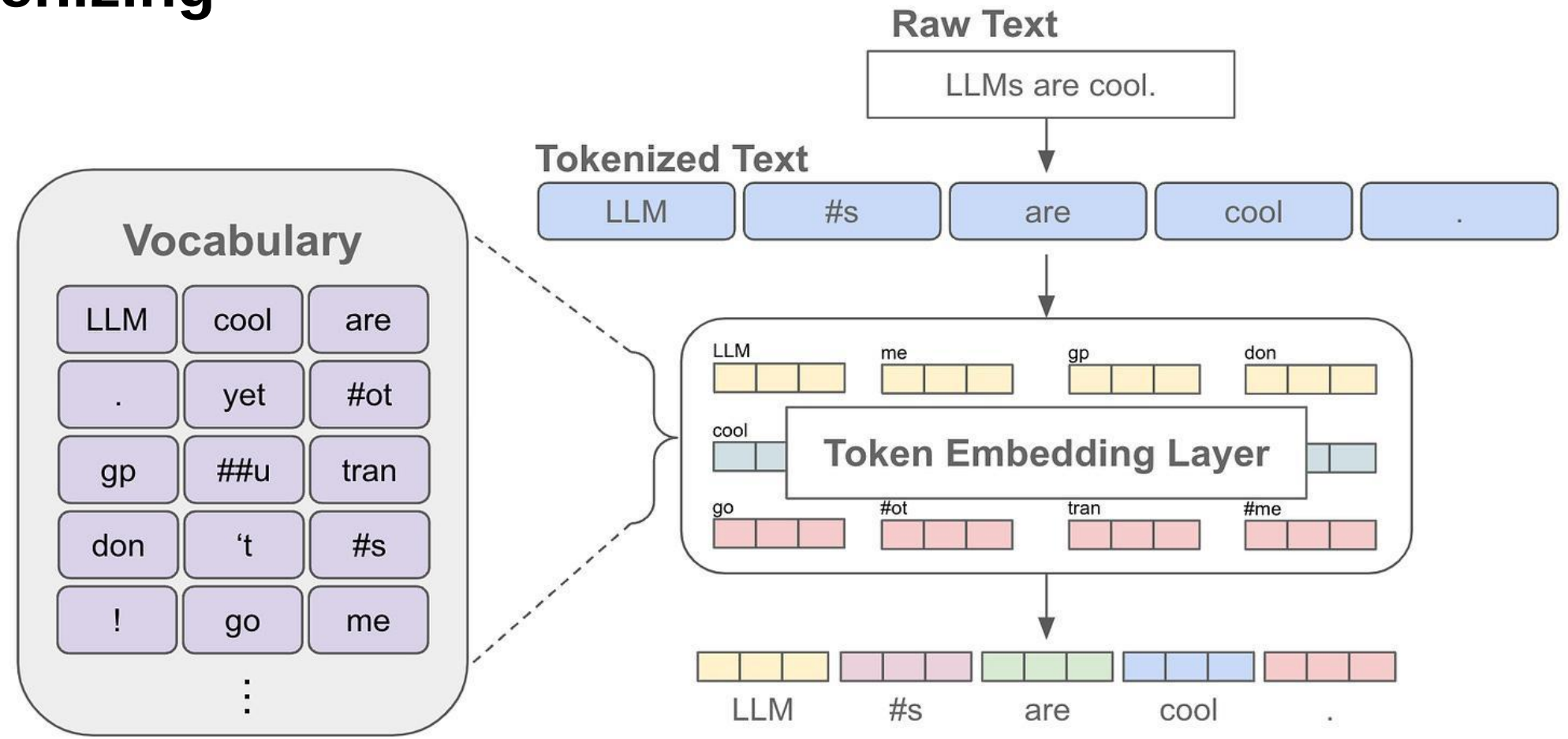
YeonHwi. You

PART 01

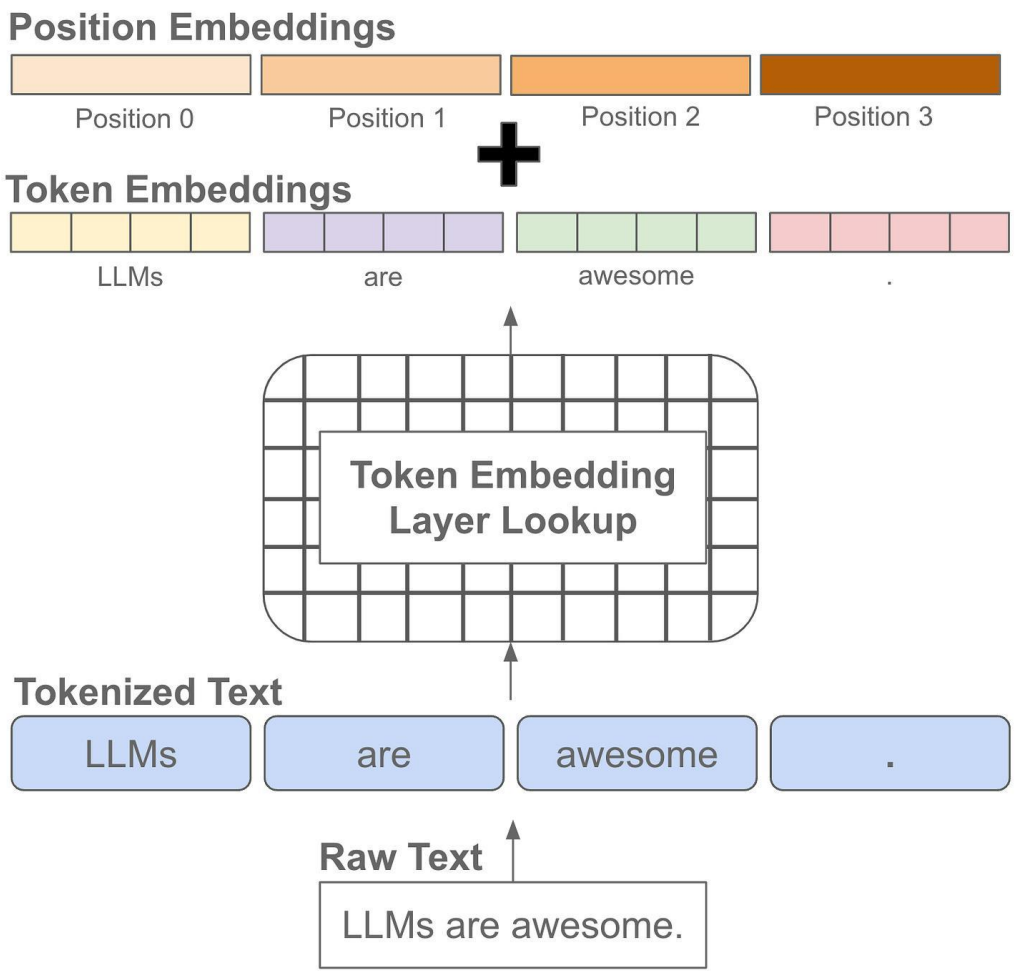
LLM 의 동작원리

YeonHwi. You

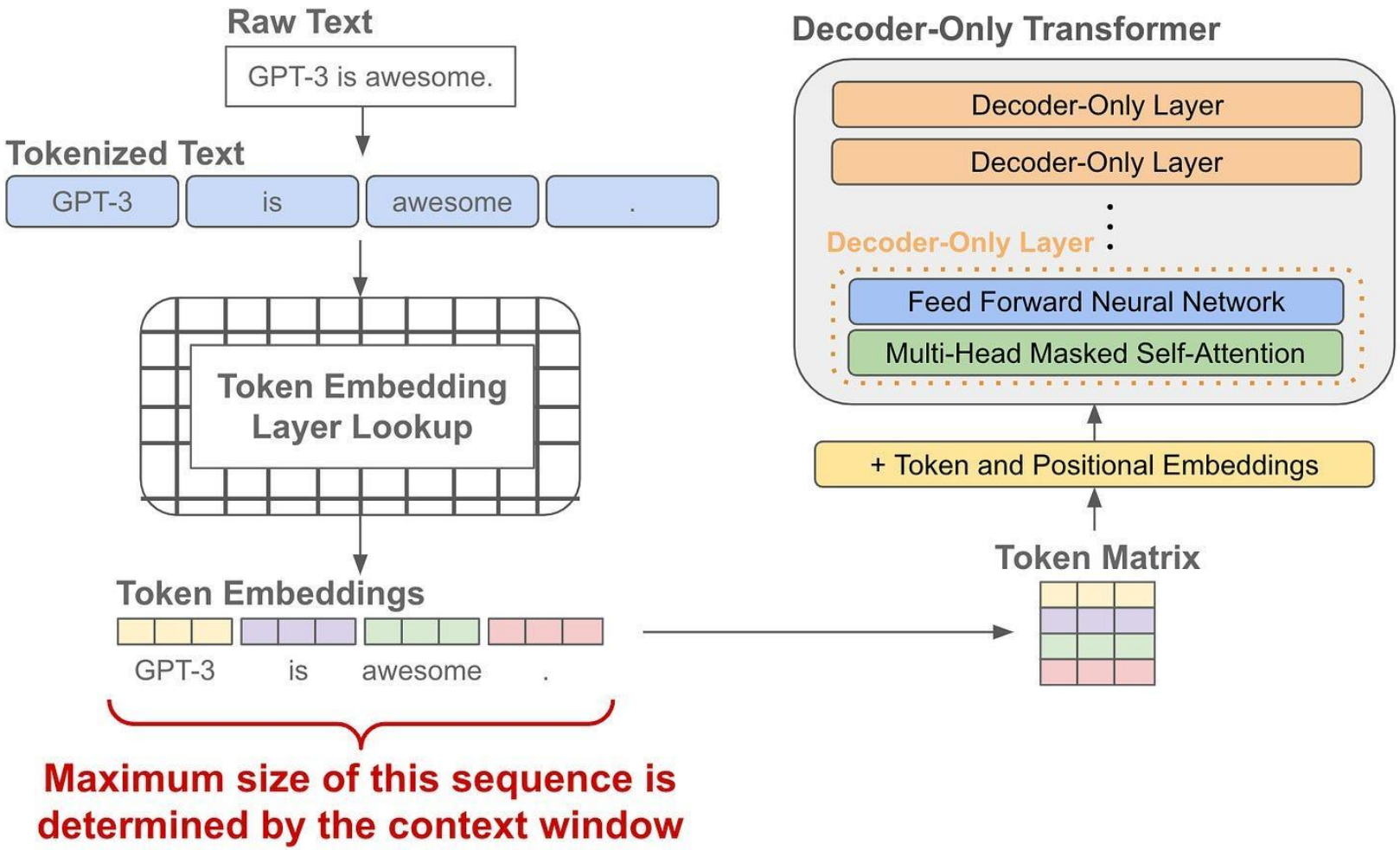
Tokenizing



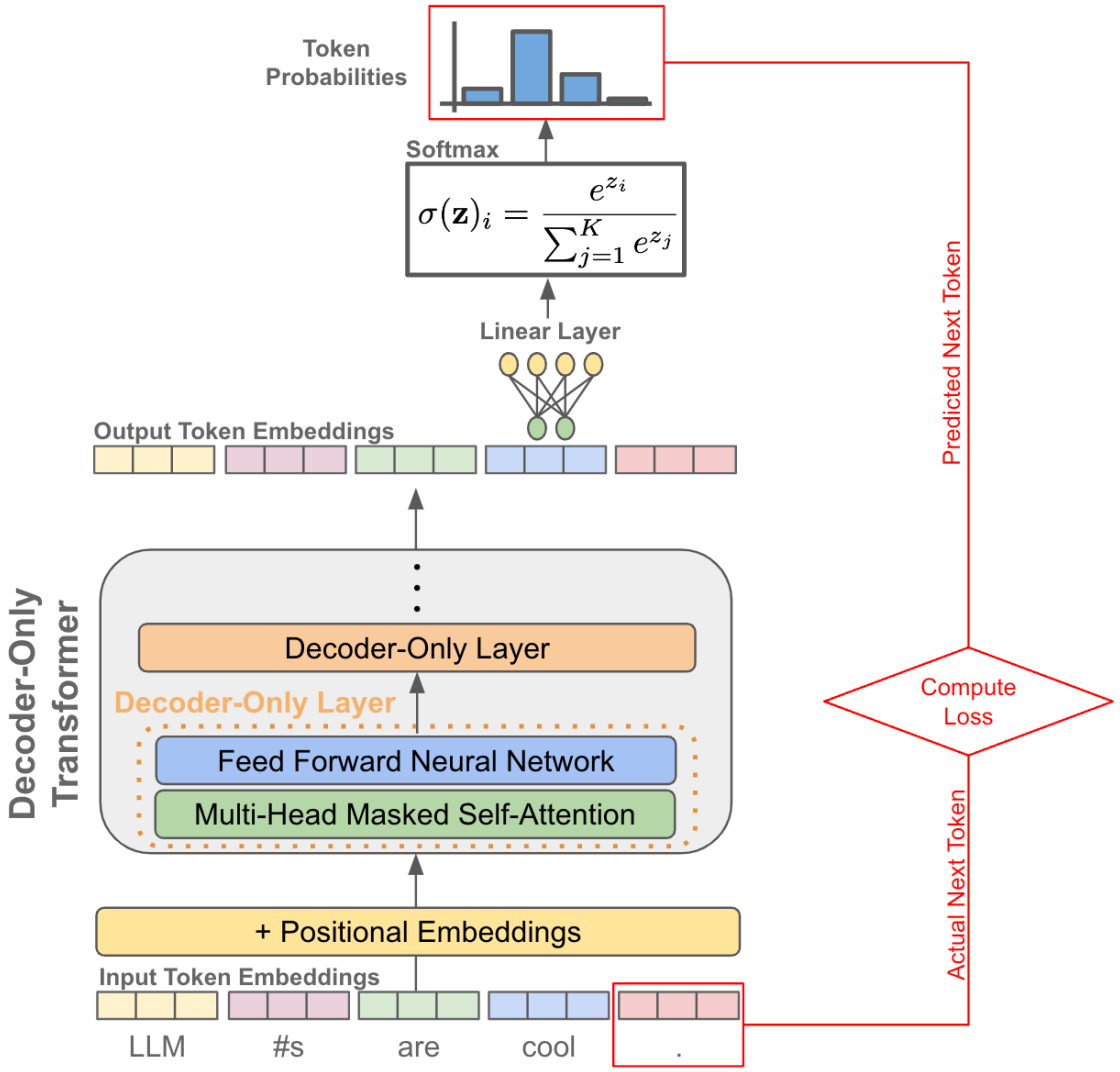
출처 – https://cameronrwolfe.substack.com/p/language-model-training-and-inference



출처 – https://cameronrwolfe.substack.com/p/language-model-training-and-inference



출처 – https://cameronrwolfe.substack.com/p/language-model-training-and-inference

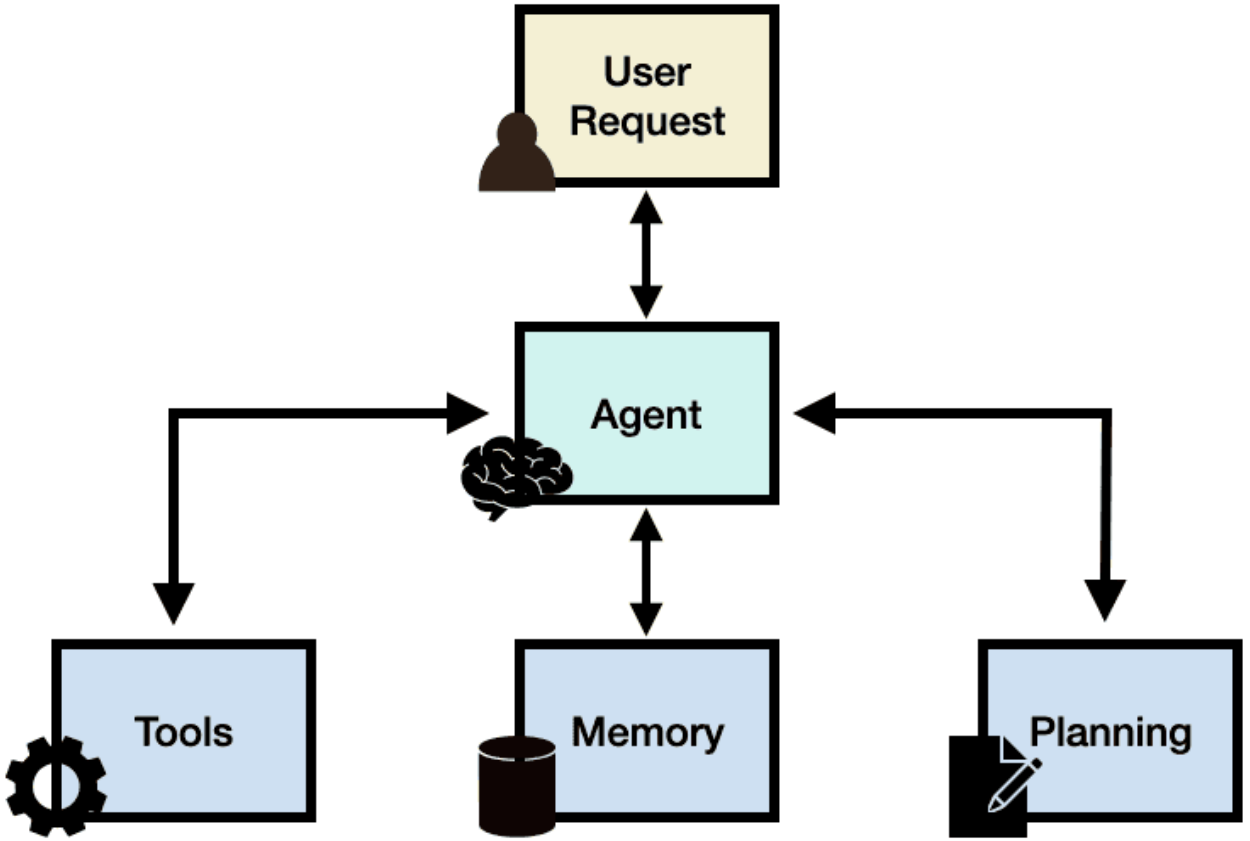


출처 – https://cameronrwolfe.substack.com/p/language-model-training-and-inference

PART 02

Agent & Multi Agent

에이전트란 ?



출처 : https://www.promptingguide.ai/kr/research/llm-agents

Agent Framework 의 구성요소

- 사용자 요청 **(User Request) -** 사용자의 질문이나 요청
- 에이전트 **(Agent)/** 두뇌 **(Brain) -** 관리자의 역할을 하는 에이전트의 핵심
- 계획 **(Planning) -** 에이전트가 미래 행동을 계획하는 것을 도움
- 메모리 **(Memory) -** 에이전트의 과거 행동을 관리

출처: https://www.promptingguide.ai/kr/research/llm-agents

Agent Framework 의 구성요소 : Planning

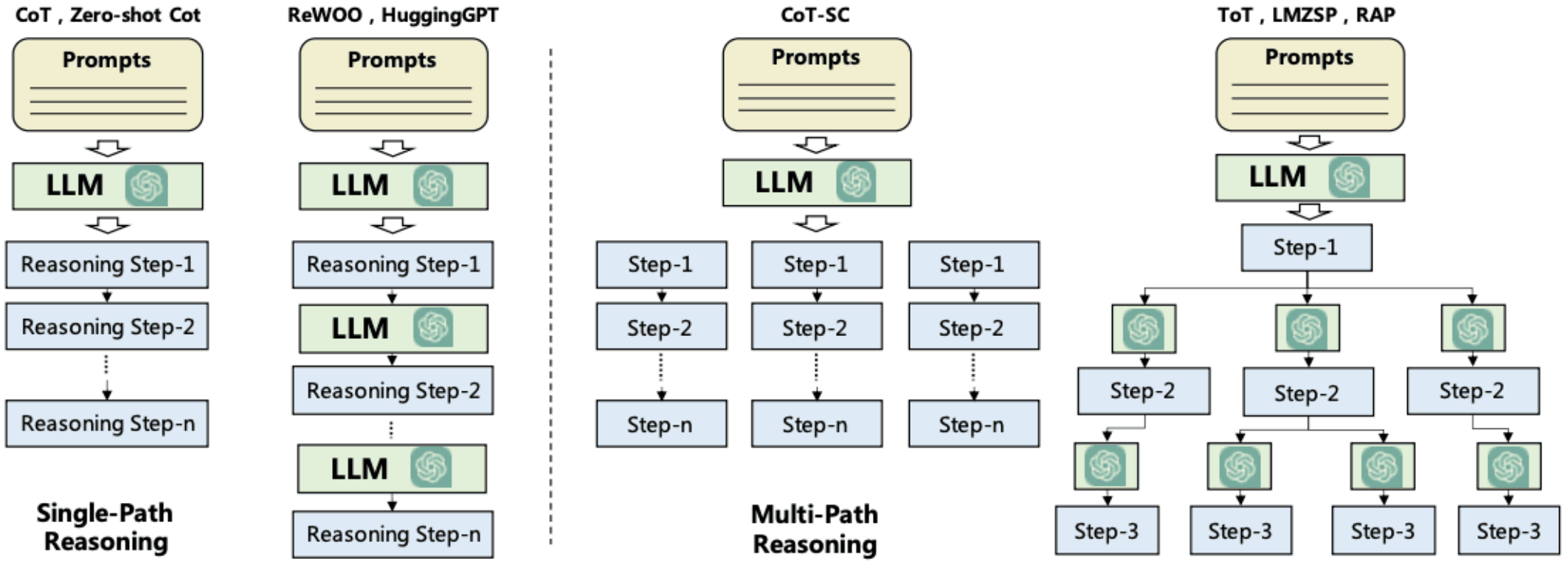
피드백없는 계획

- **Planning** 은 에이전트가 사용자의 요청에 답하기 위해 해결해야 할 단계나 작업들을 세분화하는 데 도움
- 이러한 단계는 에이전트가 문제를 더 효과적으로 추론하고 신뢰할 수 있는 해결책을 찾는 데 필요
- 계획 모듈은 LLM 을 이용하여 사용자의 질문에 도움이 되는 하위 작업을 포함한 상세한 계획
- 작업 분해에 사용되는 인기 있는 기술로는 **Chain of Thought** 와 **Tree of Thoughts** 가 있으며,

  이는 단일 경로 추론과 다중 경로 추론으로 구분가능

출처 : https://www.promptingguide.ai/kr/research/llm-agents

Agent Framework 의 구성요소 : Planning



출처 : https://www.promptingguide.ai/kr/research/llm-agents

Agent Framework 의 구성요소 : Planning

피드백있는 계획

- 계획 모듈들은 피드백이 없어 복잡한 작업에 대한 장기적인 계획을 세우는 데 어려움 존재
- 이 문제를 해결하기 위해, 모델이 과거 행동과 관찰을 바탕으로 실행 계획을 반복적으로 평가하고 조정하는 메커니즘을 사용.
- 이 방법의 목표는 과거의 실수를 수정하고 개선하여 최종 결과의 질을 높이는 것.
- ReAct 는 추론과 행동을 결합하여 LLM 이 여러 단계를 번갈아 가면서 복잡한 작업을 해결할 수 있도록 유도.
- 이 단계들은 생각, 행동, 관찰로 구성되며, ReAct 는 환경으로부터 관찰 형태의 피드백 제공.

출처 : https://www.promptingguide.ai/kr/research/llm-agents

Agent Framework 의 구성요소 : Planning

(1d) ReAct (Reason + Act)

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.
Act 1: Search[Apple Remote]
Obs 1: The Apple Remote is a remote control introduced in October 2005 by Apple … originally designed to control the Front Row media center program …

Thought 2: Apple Remote was originally designed to control the Front Row media center program. I need to search Front Row next and find what other device can control it.
Act 2: Search[Front Row]
Obs 2: Could not find [Front Row]. Similar: ['Front Row Seat to Earth', 'Front Row Motorsports',' Front Row (software) ', …]

Thought 3: Front Row is not found. I need to search Front Row (software) .
Act 3: Search[Front Row (software)]
Obs 3: Front Row is a discontinued media center software …

Thought 4: Front Row (software) is controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.
Act 4: Finish[keyboard function keys]



출처 : https://www.promptingguide.ai/kr/research/llm-agents

Agent Framework 의 구성요소 : Memory

- 메모리는 에이전트와 사용자 간의 모든 상호작용, 환경에서의 과거 생각, 행동 및 관찰을 포함하는 것을 저장하는 데 도움
- 단기 기억 **(Short-term memory)** - 에이전트의 현재 상황에 대한 컨텍스트 정보를 포함, 일반적으로 컨텍스트 윈도우의 제한으로 인해 짧고 유한한 문맥 내 학습으로 구현
- 장기 기억 **(Long-term memory)** - 에이전트의 과거 행동과 생각을 장기간 보존하고 회상해야 하는 내용을 포함, 에이전트가 필요에 따라 관련 정보를 빠르고 확장 가능한 검색을 통해 접근하는 외부 저장소를 사용
- 하이브리드 메모리 는 단기 기억과 장기 기억을 통합하여 에이전트의 장기적 추론 능력과 경험 축적 능력을 강화
- 계획과 메모리 모듈 은 에이전트가 동적 환경에서 효과적으로 작동하고 과거 행동을 잘 회상하며 미래 행동을 계획할 수 있도록

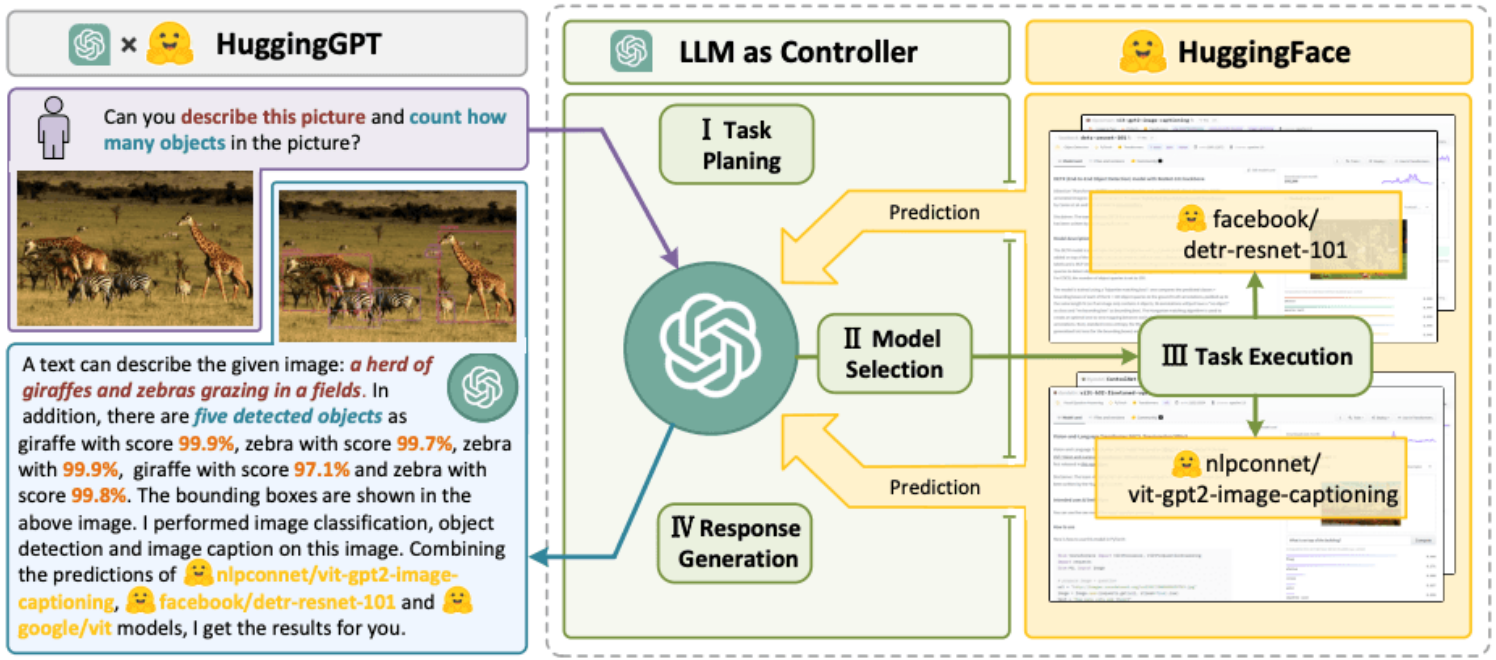
출처 : https://www.promptingguide.ai/kr/research/llm-agents

Agent Framework 의 구성요소 : Tool

도구는 LLM 에이전트가 외부 환경과 상호 작용하는 데 도움을 주는 도구나 도구 집합을 의미

위키피디아 검색API, 코드 인터프리터, 수학 엔진, 데이터베이스, 지식 베이스, 외부 모델등이 포함

에이전트가 외부 도구와 상호작용할 때요청을충족시키고 , 부분 작업을 완료하기 위해 필요한 정보를 얻는 워크플로우를 통해 작업을 수행



출처 : https://www.promptingguide.ai/kr/research/llm-agents

PART 03

RAG

YeonHwi. You

RAG (Retrieval-Augmented Generation)

RAG 의 등장 배경과 필요성 **(Why RAG?)**

1. 거대 언어 모델 **(LLM)** 의 한계

- 환각 현상 (Hallucination): LLM 이 학습 데이터에 없는 내용을 사실인 것처럼 지어내는 문제
- 지식의 최신성 부족 (Knowledge Cut-off): 특정 시점까지의 데이터로만 학습되어 최신 정보를 알지 못함 (예 : "ChatGPT 3.5 는 2022 년 1 월까지의 정보만 알고 있습니다.")
- 정보 출처의 부재: 답변의 근거를 제시하지 못해 신뢰성을 확보하기 어려움
- 높은 파인튜닝 (Fine-tuning) 비용: 새로운 지식을 학습시키기 위한 재학습/파인튜닝에 많은 시간과 비용이 소요됨.

RAG (Retrieval-Augmented Generation)

RAG 의 정의와 핵심 아이디어

정의 : 외부 지식 베이스(Knowledge Base) 에서 정보를 검색(Retrieval) 하고, 검색된 정보를 바탕으로 LLM 이 답변을 생성(Generation) 하는 하이브리드 접근 방식.

핵심 비유 : " 똑똑한 학생(LLM) 에게 오픈북(외부 지식) 을 쥐여주고 시험을 보게 하는"것

학생 혼자 시험 보면 → 아는 건 잘 풀지만, 모르는 건 상상해서 씀(LLM 단독 사용)

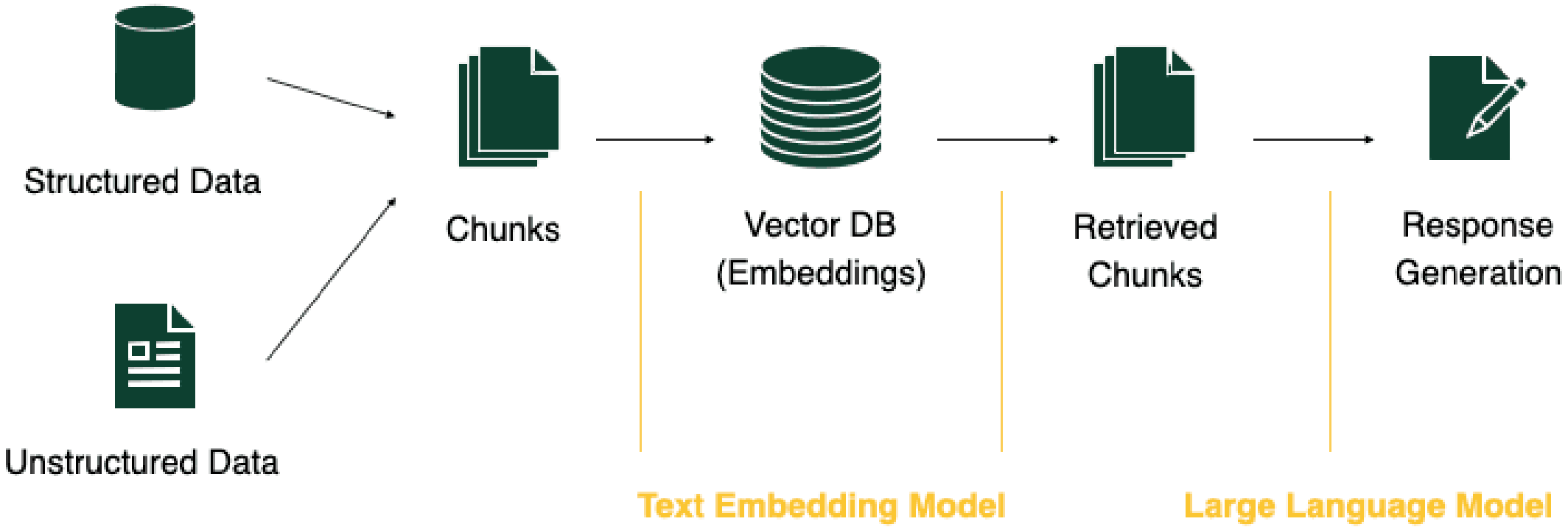
오픈북 시험을 보면 → 책에서 정확한 근거를 찾아 답변함(RAG)

RAG 를 사용해야 하는 이유

- 정확성 및 신뢰성 향상 : 검증된 외부 데이터를 근거로 답변하므로 환각 현상을 획기적으로 줄임
- 최신 정보 반영 : 외부 지식 베이스만 업데이트하면 되므로 LLM 을 재학습할 필요 없이 최신 정보 반영이 가능함
- 출처 제시 가능 : 어떤 문서를 참고하여 답변했는지 출처를 제시할 수 있어 사용자가 신뢰할 수 있음
- 비용 효율성 : 매번 모델을 파인튜닝하는 것보다 훨씬 경제적이고 빠름
- 보안 및 프라이버시 : 특정 내부 문서나 민감한 정보에 대해서만 답변하도록 제어 가능

출처 https://khw063011.tistory.com/105

Simple RAG


Structured Data
Unstructured Data
Chunks
Vector DB
(Embeddings)
Retrieved
Chunks
Response
Generation
Text Embedding Model
Large Language Model

RAG 의 핵심 프로세싱 1

핵심 프로세스 1: 데이터 인덱싱 (Indexing - Offline Process)

- 목표 : 우리의 지식 베이스(문서, DB 등)를 AI 가 검색하기 좋은 형태로 미리 가공하여 저장하는 과정.
- **Step 1:** 데이터 로딩 (Data Loading): PDF, TXT, HTML, Notion, DB 등 다양한 소스에서 원본 데이터를 불러옴.
- **Step 2:** 데이터 분할 (Chunking/Splitting): 큰 문서를 의미 있는 작은 단위(Chunk)로 나눔.
  - 왜? LLM 의 컨텍스트 길이(Context Length) 한계, 검색 정확도 향상을 위해.
  - 방법: 고정 크기 분할, 재귀적 문자 분할 등.
- **Step 3:** 임베딩 (Embedding): 각 텍스트 조각(Chunk)을 임베딩 모델을 사용해 숫자 벡터(Vector)로 변환.
  - 임베딩이란? 텍스트의 의미를 다차원 공간의 한 점(좌표)으로 표현하는 기술. 의미가 비슷한 텍스트는 공간상에서 가까운 위치에 존재함.
- **Step 4:** 벡터 저장소 저장 (Vector Store): 생성된 벡터를 벡터 데이터베이스(Vector DB)에 원본 텍스트와 함께 저장.
  - 역할: 수많은 벡터 중에서 특정 벡터와 가장 유사한 벡터들을 초고속으로 검색.
  - 예시: FAISS, ChromaDB, Pinecone 등.

RAG 의 핵심 프로세싱 2

핵심 프로세스 **2:** 검색 및 생성 **(Retrieval & Generation - Online Process)**

- **Step 1:** 사용자 질문 **(User Query)**
- **Step 2:** 질문 임베딩 **(Query Embedding):** 사용자의 질문도 인덱싱 때와 동일한 임베딩 모델을 사용해 벡터로 변환.
- **Step 3:** 유사도 검색 **(Similarity Search):** 질문 벡터와 벡터DB 에 저장된 문서 벡터들 간의 유사도를 계산하여 가장 관련성 높은 상위K 개의 문서 조각(Top-K Chunks) 을 검색.
- **Step 4:** 프롬프트 구성 **(Prompt Augmentation):** 검색된 문서 조각(Context) 과 사용자의 질문(Query) 을 정해진 템플릿에 따라 하나의 프롬프트로 결합
- **Step 5: LLM** 답변 생성 **(LLM Generation):** 구성된 프롬프트를LLM 에 입력하여 최종 답변을 생성

프롬프트 예시

[Context]
{검색된 문서 내용1}
{검색된 문서 내용2}
...

[Question] {사용자의 원본 질문}

위 Context 를 바탕으로 Question 에 대한 답변을 생성해주세요 만약 Context 에 답이 없다면 "정보를 찾을 수 없습니다" 라고 답변해주세요

RAG의 핵심 구성 요소 1

1. 임베딩 모델 (Embedding Models)

•역할: 텍스트의 '의미'를 벡터로 변환하는 엔진
•종류: OpenAI text-embedding-ada-002, BGE (BAAI General Embedding), 한국어 특화 ko-sroberta 등.
•선택 기준: 성능, 비용, 지원 언어, 임베딩 차원 수.

2. 벡터 데이터베이스 (Vector DBs)

•역할: 벡터를 효율적으로 저장하고 빠르게 검색하는 시스템.
•핵심 기술: ANN (Approximate Nearest Neighbor, 근사 최근접 이웃) 알고리즘을 사용해 속도 확보.
•종류:
•In-Memory 라이브러리: FAISS, ScaNN
•독립 실행형 DB: ChromaDB, Weaviate, Qdrant
•클라우드 서비스형 Pinecone, Zilliz Cloud

RAG 의 핵심 구성 요소 2

3. 리트리버 **(Retriever)**

•역할: 가장 관련성 높은 문서를 '잘' 찾아오는 전략.

•**Semantic Search (** 의미 기반 검색 **):** 지금까지 설명한 벡터 유사도 검색.

•**Keyword Search (** 키워드 기반 검색 **):** 전통적인 검색 방식 (예: BM25). 특정 용어나 고유명사 검색에 강점.

•**Hybrid Search (** 하이브리드 검색 **):** Semantic Search 와 Keyword Search 를 결합하여 두 방식의 장점을 모두 취하는 가장 효과적인 방법.

4. 생성 모델 **(Generator - LLM)**

•역할: 주어진 문맥(Context) 을 바탕으로 자연스러운 답변을 생성.

•선택 기준: 답변 생성 능력, 추론 능력, 비용, 속도, 컨텍스트 창 크기.

•종류: GPT-4, GPT-3.5-Turbo, Claude 3, Llama3 등.

고급 RAG 기법 및 트렌드

1. 검색 품질 향상을 위한 기법

- **Query Transformation (** 질의 변환 **):** 사용자의 질문을 검색에 더 유리한 형태로 변환 (예: 여러 개의 하위 질문으로 분리, 가상의 답변을 생성 후 검색)
- **Re-ranking (** 재순위화 **):** 1 차로 검색된 문서들을 더 정교한 모델 (Re-ranker) 을 사용해 순위를 다시 매겨 정확도를 높임.

2. 답변 품질 향상을 위한 기법

- **Small-to-Large (Chunk-to-Document):** 처음에는 작은 Chunk 로 검색하여 관련 문서를 찾고, 해당 문서 전체를 LLM 에 제공하여 더 넓은 문맥으로 답변을 생성

3. Agentic RAG / Self-Corrective RAG

- LLM 이 스스로 판단하여 검색된 정보가 충분하지 않으면 추가 검색을 하거나, 질문을 재구성하여 검색어를 사용하는 등 능동적으로 행동하는 RAG.

4. Multi-modal RAG

- 텍스트뿐만 아니라 이미지, 표, 차트 등 다양한 형태의 데이터를 검색하고 답변에 활용하는 RAG.

PART 04

LLM Application

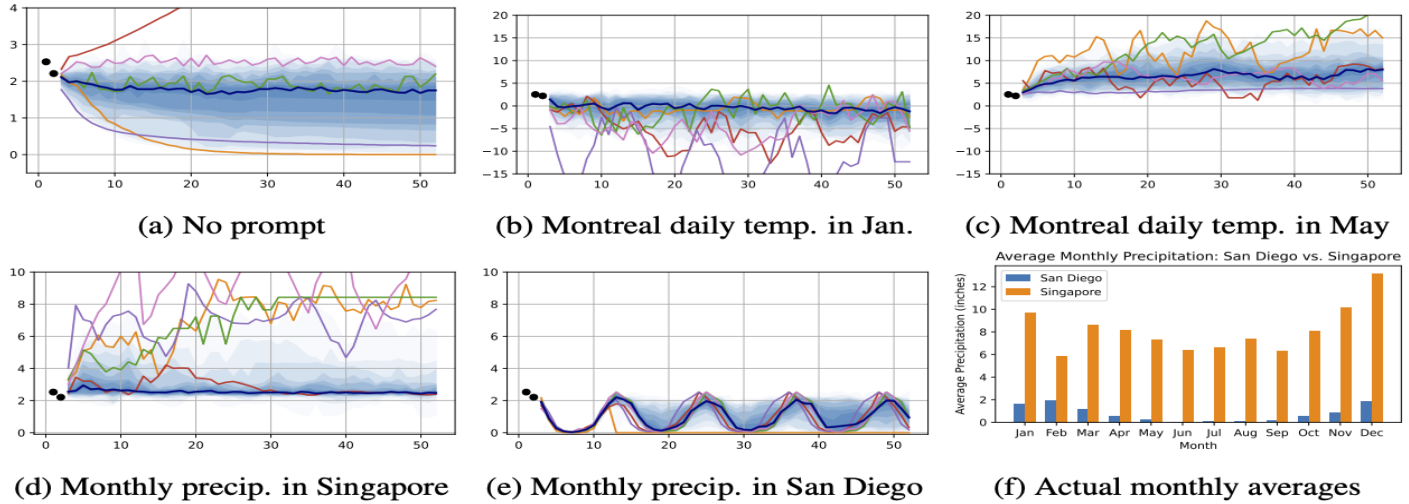
YeonHwi. You

프롬프트 **(prompt)** 란 **?**



- AI 모델에게 원하는 작업을 수행하도록 지시하는 명령어 또는 질문.
- 단순한 질문부터 복잡한 코드 생성 요청까지 모든 것이 프롬프트가 될 수 있음
- 예시: " 오늘 서울 날씨 어때?" (단순) vs " 파이썬으로 웹사이트에서 최신 뉴스 기사 제목을 크롤링하는 코드 짜줘" (복잡)
- 프롬프트는 보통 다음과 같은 구성으로 이루어짐
  - Instruction(지시 **):** 모델이 수행할 작업이나 유저의 요청 사항
  - Context(맥락 **):** 모델이 보다 나은 응답을 도출하기 위한 외부의 정보 혹은 추가적인 맥락
  - Input Data(입력 데이터 **):** 응답을 요청하는 질문이나 입력 데이터
  - Output Indicator(출력 지표 **):** 출력의 형식이나 유형

Prompt 의 중요성



- Prompt 는 모델이 학습한 어떠한 정보값을 이끌어내는데 매우 중요한 역할을 하고 있음
  LLM Processes: Numerical Predictive Distributions Conditioned on Natural Language(Requeima et al., 2024)
- 위의 논문은 기존의 LLM 이 학습한 어떠한 정보 값을 특정 토큰과 데이터를 통해서 이끌어내 이를 검증한 논문(University of Toronto, Cambridge)
- 이처럼 Prompt 를 잘 정의하고 활용하는 것은 기존 모델의 성능을 향상시킬 수 있으며, 이에 이를 통해서 기존 모델이 학습한 정보 활용 가능

출처 – LLM Process(Requeima et al., 2024)

Prompt Template: Alpaca



- **Stanford Alpaca** 프로젝트에서 유래된 템플릿으로, GPT 기반의 모델을 Instruction Tuning 을 통해 개선한 방식, 주로 사용자 질문에 대한 대답을 유도하거나, 모델이 주어진 명령어에 맞는 답을 생성하도록 설계된 템플릿

Alpaca_chat: 대화형 챗봇과 같은 상황에서 사용되는 템플릿으로 명령어 및 사용자 질문에 대한 응답을 생성하도록 도와줌

alpaca_chat.load_qa: 질문-응답 태스크에 사용되며, 질문에 대한 대답을 정확히 제공하는 데 초점을 맞춘 템플릿

alpaca_chat.load_camel_ai: Camel AI 와 연동된 대화형 프롬프트를 처리하는 템플릿

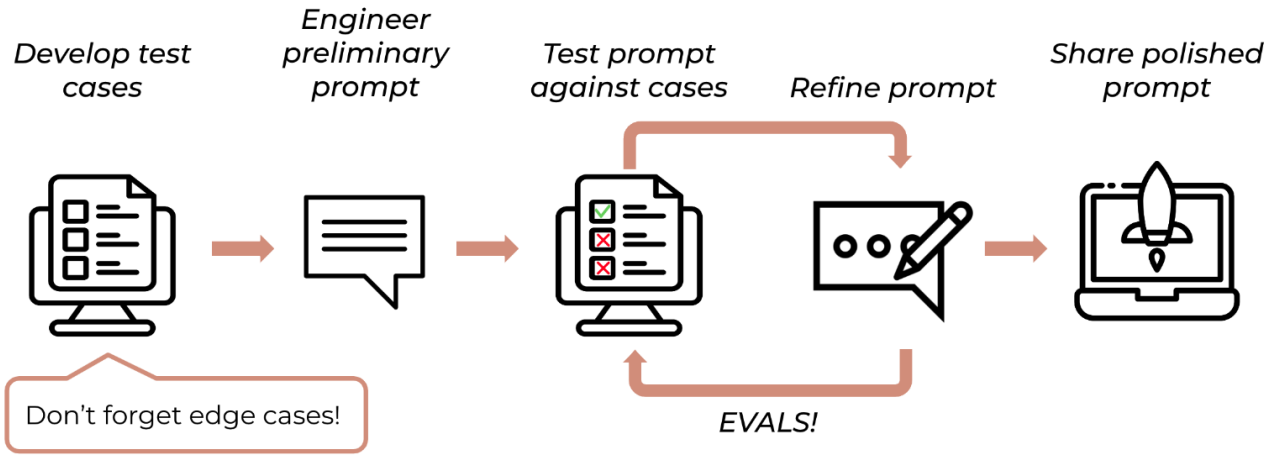
alpaca_w_system.load_open_orca: OpenOrca 와 연계된 시스템에서 사용하는 템플릿으로, OpenOrca 데이터셋을 활용해 더욱 다양한 데이터 학습을 지원

```
"prompt_input":
"Below is an instruction that describes a task, paired with an input that provides further context. Write a response that appropriately completes the request.\n\n
### Instruction:\n{instruction}\n\n### Input:\n{input}\n\n
### Response:\n",

"prompt_no_input":
"Below is an instruction that describes a task. Write a response that appropriately completes the request.\n\n
### Instruction:\n{instruction}\n\n
### Response:\n",
```

출처 – https://crfm.stanford.edu/2023/03/13/alpaca.html

프롬프트 엔지니어링의 개념과 중요성



프롬프트 엔지니어링은 생성형 AI 모델이 원하는 결과를 내도록 하는 과정으로, 명확하고 구체적인 입력 텍스트를 설계하여 모델이 예상대로 작동하도록 하는 기술

AI 와의 효과적인 소통 방법 **:**

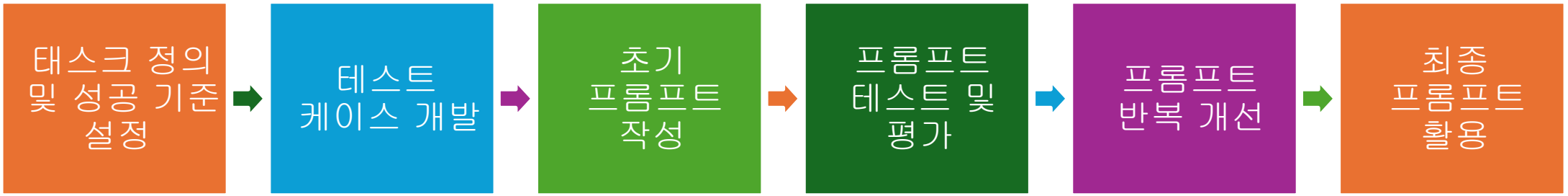
- 생성형 AI 는 사용자가 제시하는 프롬프트를 통해 작동되므로, 따라서 명확하고 구체적인 프롬프트를 작성하는 것이 AI 가 원하는 결과물을 정확하게 이해하고 생성하도록 이끄는 핵심 역량

정확하고 원하는 결과를 얻기 위한 프롬프트 작성의 중요성 **:**

- 모호하거나 불분명한 프롬프트는 AI 가 의도하지 않은 방향으로 답변하거나, 품질이 낮은 결과를 생성, 효과적인 프롬프트는 AI 의 잠재력을 최대한 활용하여 높은 품질의 결과물을 얻는 데 필수적

출처: https://www.magicaiprompts.com/blog/2024/04/21/claude-prompt-engineering-complete-guide

프롬프트 엔지니어링의 과정



Zero-shot Prompting

- **Zero-shot Prompting:** 모델에게 추가적인 예시나 맥락 없이 오직 지시사항만으로 결과 생성을 요청하는 기
- 장점
  - 빠르고 간편함
  - 긴 문맥으로 인해 생기는 혼란 감소
- 단점
  - 모델이 의도한 결과를 정확히 이해하지 못할 수 있음
  - 복잡한 작업에서 성능이 저하될 가능성 높음
- 예시:
- 다음 문장을 영어로 번역하세요: "안녕하세요, 오늘 날씨가 참 좋네요"
- 명령은 구체적이고 간결하게 작성할 것
- 수행 과제가 단순한 경우에는 Zero-shot 이 효과적

Few-shot Prompting

- Few-shot: 프롬프트에 몇 가지 예시를 포함하여 모델에게 원하는 형식을 직접 학습시키는 방식
- 장점
  - 복잡한 작업에서도 정확도를 높임
  - 모델이 명확한 문맥과 패턴을 이해하도록 도와줌
- 단점
  - 프롬프트가 길어지며 비용과 처리 시간 증가 가능
  - 부적절한 예시가 포함되면 성능이 오히려 떨어질 수 있음
- 포함되는 예시는 대표적이고 명확한 사례를 선정
- 예시 간의 형식과 구조를 통일하면 성능 향상 가능

```
문장을 감정(긍정, 부정)으로 분류하세요.

문장: "이 제품 정말 최고예요!"
감정: 긍정

문장: "서비스가 너무 느리고 불친절했어요."
감정: 부정

문장: "맛있고 친절한 가게였어요!"
감정:
```

In-Context Learning

- In-Context Learning (ICL) 의 정의
  - In-Context Learning 은 AI 모델이 학습 데이터를 추가적인 훈련 없이, 오직 프롬프트 내에 제공된 예시를 통해 작업을 즉각적으로 학습하고 이해하는 능력
  - 제시된 문맥(context) 을 기반으로 사전학습된 정보를 바탕으로 즉시 작업을 수행

- ICL 의 원리
  - LLM 은 이미 방대한 데이터를 기반으로 다양한 언어 패턴과 규칙을 내재화
  - 프롬프트에 제시된 예시들은 이 모델이 내재화한 지식을 activate 하여 특정 작업의 패턴을 인식하고 일반화
  - 이는 마치 '문맥 안에서 즉시 fine-tuning' 이 이루어지는 것처럼 작동
- Few-shot 프롬프트에서 제시된 소수의 예시들은 모델에게 작업의 형태와 스타일, 원하는 답변의 형태를 명확히 제시
- 이러한 문맥적 예시는 모델 내부의 관련 지식을 더 쉽게 불러오도록 **(prime)** 유도
- 중요성 : 추가 학습이나 미세조정이 불필요하여 시간과 비용 절약
- 한계 : 지나치게 복잡하거나 명확하지 않은 예시들은 오히려 성능을 떨어뜨릴 수 있음

출처: https://www.magicaiprompts.com/blog/2024/04/21/claude-prompt-engineering-complete-guide

PART 05

RAG 심화 : 설계 · 디버깅 · 평가

YeonHwi. You

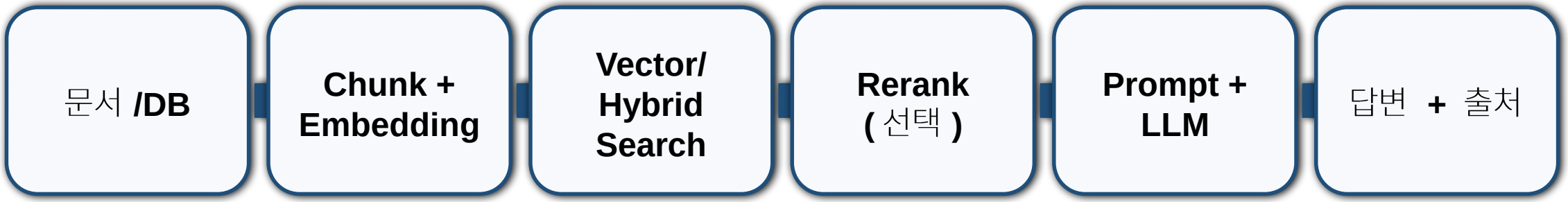
RAG 의 디버깅 포인트

"RAG 는 개념보다 튜닝 **/** 디버깅이 핵심 **"**

- RAG 큰 그림: 검색(근거 선택) + 생성(서술)
- 인덱싱 품질: 문서 정제 · Chunking · Metadata
- 검색 품질: Retriever 전략 · Hybrid · Reranker
- 답변 품질: 컨텍스트 구성· 출처 표기 · 거부/ 불확실성
- 실패 모드& 디버깅 체크리스트
- 평가(offline/online) 와 운영(보안, 업데이트, 모니터링)

RAG 를 한 문장으로

"LLM 이 답을 쓰고, Retriever 가 근거를 고르는 구조



핵심은검색품질 : LLM 이 똑똑해도 근거가 틀리면 답이 틀림

따라서 " 모델 성능" 보다 " 데이터/ 검색 설계" 가 체감 품질을 좌우

RAG vs Fine-tuning vs Tool-use

언제 무엇을 쓰는가?

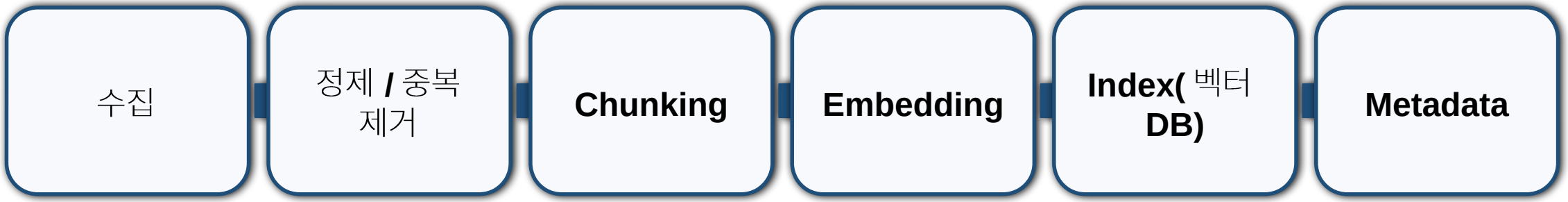
RAG
- 지식/문서 기반 Q&A
- 근거·출처 제시
- 문서 업데이트 쉬움

Fine-tuning
- 스타일/포맷/정책 학습
- 반복 작업의 일관성
- 추론 습관 교정

Tool-use(함수/DB 질의)
- 정형 데이터 조회
- 정확한 계산/검색
- 업무 자동화

인덱싱 파이프라인 : "RAG 의 절반 "

Offline(사전 작업) 품질이 검색 품질을 결정

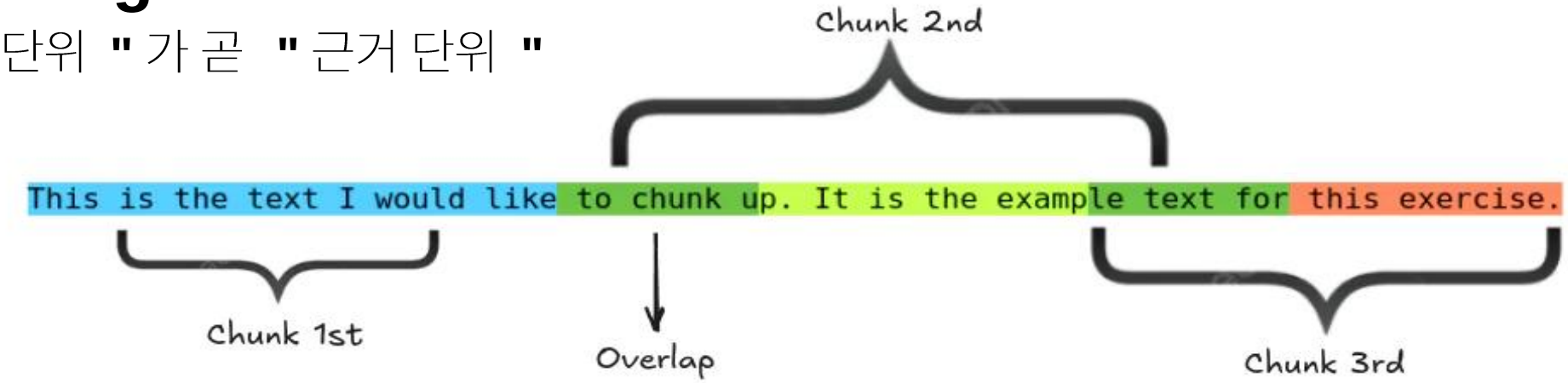


정제 예: 머리말/ 꼬리말/ 목차 제거, 표 / 코드 블록 처리, 중복 문서 제거

Metadata 예 : 문서명, 섹션, 날짜, 버전, 권한(ACL), URL/ 파일경로

Chunking 이 왜 이렇게 중요할까 ?

" 검색 단위 " 가 곧 " 근거 단위 "



- 너무 크면: 검색은 맞아도 답변에 필요한 부분만 추출/활용이 어려움(노이즈↑)
- 너무 작으면: 문맥이 잘려 의미가 깨짐(근거가 파편화)
- Chunking 은 검색 Recall/Precision, 비용(토큰), 출처 표기까지 모두에 영향
- 권장 접근: " 시작값" 을 정하고→ 실패 사례로 튜닝

Chunking 시작값 가이드

정답은 없지만, 출발점은 필요

	Score
1. No Chunk	3.0769
2. Token Splitter	3.2308
3. Recursive	3.3077
4. Window	3.3590
5. Semantic	3.5128

- 일반 문서 **(Q&A):** 250~500 tokens / overlap 50~100 tokens 시작
- **FAQ/** 짧은 규정 **:** 한 항목(질문 - 답) 단위로 그대로chunk
- 매뉴얼 **/** 정책 **:** 제목(Heading) 기반 + 길면 추가로 토큰 기준 분할
- 표 **/** 리스트 **:** 행 / 항목 단위로 의미가 유지되게 분(할필요하면 표를 텍스트로 정규)화
- 각 chunk 에 " 문서명 · 섹션 · 페이지 " 를 함께 저장(출처 / 디버깅에 필수)

Chunking 전략 옵션

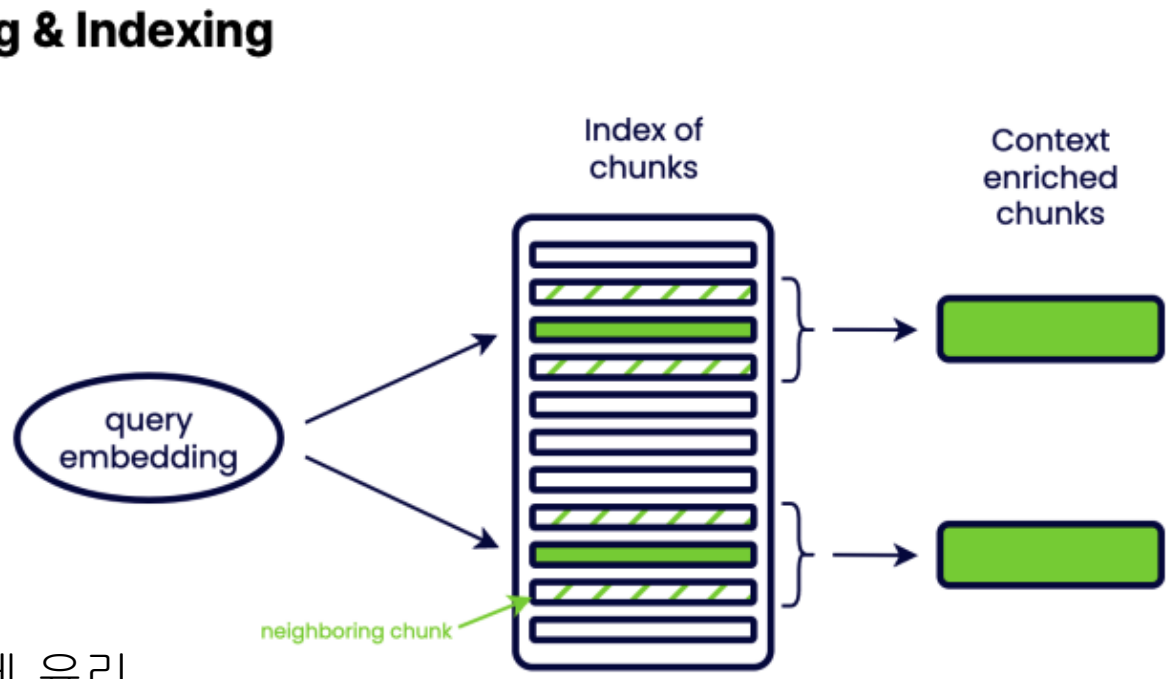
고정 분할만이 답은 아님

- 고정 길이 분할
- 구현 쉬움, 기준점으로 좋음
- 문단/제목 경계가 깨질 수 있음

- 구조 기반 분할 **(Heading/ 문단 / 조항)**
- 정책/매뉴얼 문서에서 효과적

- 의미 기반 분할 **(Semantic chunking)**
- 문장/문단 유사도 변화를 기준으로 끊음, 문맥 보존에 유리

- 계층형 인덱싱 **(Chunk→Section→Doc)**
- Small-to-Large, 요약 노드 등과 결합 가능



Metadata 설계 : " 필터 " 와 " 출처 " 를 만든다

Vector search 만으로는 운영이 어렵다

- 필수 메타데이터 : doc_id, title, section, page, created_at/updated_at, source_url
- 권한 / 보안 : user_role, department, clearance_level 등 (ACL 필터링)
- 검색 개선 : doc_type(FAQ/ 정책 / 매뉴얼), product_name, tag
- 운영 : version, ingestion_time, embedding_model, chunk_size

임베딩 모델 선택 기준

"LLM 이 아니라 **Retriever** 의 눈 "

- 언어 / 도메인 : 한국어· 금융· 사내 용어에서 의미가 잘 유지되는가?
- 문서 **vs** 질의 : short query 에 강한지 / long passage 에 강한지
- 차원 **(d):** 메모리· 속도· 정확도 trade-off
- 정규화 여부 : cosine/dot 제품 설계에 영향
- 추론 비용 : TPS, 배치 임베딩, GPU/CPU 가능 여부

가장 적절한 임베딩모델의 선택은 RAG 의 성능을 높게 향상시킨다!

유사도 **(metric)** 기본기

cosine / dot / L2 차이를 알아야 튜닝이 된다

- **Cosine:** 각도 기반(벡터 길이 무시) → 가장 흔한 기본값
- **Dot product:** 길이까지 포함 → 임베딩이 정규화되어 있으면 cosine 과 동일해짐
- **L2 distance:** 거리 기반 → 일부 인덱스/라이브러리 기본값
- 중요 : **VectorDB/** 라이브러리의 기본 **metric** 을 확인 (모델 권장과 일치시키기)

Retriever 튜닝 노브 (Knobs)

체감 품질을 가장 빨리 바꾸는 레버

- **top-k:** 너무 작으면 누락(Recall↓), 너무 크면 노이즈+비용↑
- **MMR(다양성):** 비슷한 chunk 만 모이는 현상 방지
- **Metadata filter:** 기간/버전/부서/상품 등으로 후보군 축소
- **Hybrid search:** BM25(키워드) + Vector(의미) 조합
- **Query expansion:** 동의어/약어 확장, multi-query

Hybrid Search 가 강한 순간

" 고유명사 " 와 " 의미 " 를 동시에 잡는다

고유명사 / 코드명 / 약어 : 키워드 **(BM25)** 가 강함 (예 : 상품 코드, 조항 번호)

의미 유사 질문 : 벡터 검색 이 강함 (표현이 달라져도 찾음)

실전 : 1) BM25 로 후보 생성 → 2) 벡터로 점수 결합/필터 → 3) rerank

Query Transformation(질의 변환)

질문을 " 검색 잘 되는 질문 " 으로 바꾼다

- **Multi-query:** 질문을 여러 표현으로 재작성 → 각각 검색 → 결과 합치기
- **Sub-question:** 복합 질문을 하위 질문으로 분해
- **HyDE** 스타일 **:** " 가상 답 " 을 먼저 만들어 그 텍스트로 검색 (의미 힌트 강화)
- 주의 **:** 재작성은 비용 / 지연시간 증가 → 캐싱 / 옵션화 권장

Re-ranking: " 읽고 판단 " 을 추가한다

1 차 검색은 빠르게, 2 차는 정확하게



- Bi-encoder(임베딩) 만으로는 " 정밀 판단" 이 어려운 케이스가 많음
- Re-ranker 는 query 와 passage 를 함께 보고 점수화 → Precision 크게 개선
- 대신 비용↑ → 보통 Top-50 이하에만 적용

컨텍스트 구성 **(Prompt Augmentation)**

"많이 넣기"가 아니라 "잘 넣기"

- **Context window** 제한 **:** k 를 키우면 토큰 비용과 답변 품질(노이즈) 이 같이 움직임
- **Chunk** 정렬 **:** (1) rerank 점수, (2) 문서 내 위치, (3) 최신 버전 우선 등 규칙화
- 중복 제거 **:** 유사 chunk 는 하나만 남기기
- **Context compression:** 요약/ 하이라이트(근거 문장) 만 추출해 넣기(선택)

RAG 프롬프트 템플릿 예시

모델이 "근거"와 "추측"을 구분하게 만들기

[System]
너는 주어진 근거 (Context) 만 사용해 답변한다.
근거에 없으면 '정보를 찾을 수 없습니다' 라고 답한다.

[Context]
(1) {chunk_1} [source: docA#p12]
(2) {chunk_2} [source: docB#sec3]

[Question]
{user_question}

[Answer format]
- 결론(1~2 문장)
- 근거(인용/ 요약 + source 표시)
- 예외/ 주의사항(있으면)

출처 표기 **(grounding)** 디자인

사용자 신뢰+ 디버깅 + 책임 추적

- 최소 단위 : chunk 마다 source_id(doc_id, page, section) 를 붙여 prompt 에 전달
- **UI/** 리포트 : 답변 하단에 " 참고 문서" 목록 제공(링크/ 페이지)
- 충돌 처리 : 서로 다른 문서가 상충하면 최신/ 권위 기준을 명시하고 둘 다 제시
- 법 / 규정 : 원문 인용(짧게) + 요약을 함께 제공하면 분쟁 리스크 감소

RAG 에서 가장 흔한 실패 6 가지

" 모델이 못하는 게 아니라 파이프라인이 안 맞는 경우

1) 정답 chunk 가 아예 인덱스에 없음 (수집 / 정제 / 업데이트 문제)

2) Chunk 가 잘못 끊겨 의미가 깨짐 (Chunking 문제)

3) Query 표현이 달라 검색이 안 됨 (질의 변환 / 하이브리드 필요)

4) top-k 가 너무 작거나 metric/index 가 부적절함 (검색 설정 문제)

5) 가져온 근거는 맞는데 LLM 이 무시함 (프롬프트 / 컨텍스트 과다)

6) 문서에 악성 지시문 포함 (prompt injection) → 보안 설계 필요

디버깅 체크리스트

"검색이 맞나?" 부터 분리해서 본다



- 로그에 반드시 남길 것query, retrieved chunk ids, scores, prompt 길이, 모델 응답
- 튜닝 순서 추천 Chunking → Retriever(top-k/MMR/Hybrid) → Rerank → Prompt

평가 : " 좋아 보인다 " 를 숫자로 바꾸기

offline + online 둘 다 필요

- **Offline(** 사전 **):** 질문 - 정답 - 근거 세트를 30~100 개라도 만들기
- **Retrieval** 지표 **:** Recall@k, MRR(정답 근거가 상위에 뜨는가?)
- **Answer** 지표 **:** 정답성, 근거 일치(faithfulness), 출처 누락률
- **Online(** 운영 **):** 사용자 만족/ 재질문율, 클릭한 출처, 응답 시간, 비용

혹은 적절한 평가 데이터셋을 기반 으로 **RAG** 시스템을 지속적으로 평가 하고 **Tuning** 하는 것이 중요!

운영 아키텍처 체크포인트

"챗봇"이 아니라 "검색 서비스"에 가깝다

- 인덱싱 파이프라인: 정기 업데이트, 실패 재처리, 버전 관리
- 캐싱: (1) query→retrieval 결과, (2) embedding 캐시, (3) 최종 답변 캐시
- 모니터링: top-k 히트율, 빈 검색, 응답 길이/토큰, 에러율
- 데이터 거버넌스: 삭제 요청(Right to be forgotten), PII 마스킹, 접근 제어